
GUIDE 03

LLMOps & MLOps for Agentic AI

A comprehensive operational guide for production AI agent systems

Agentic AI Builder Expert Bootcamp

Batch 4.0 | April 2026

Table of Contents

1. What is LLMOps vs MLOps?
2. The Agentic AI Operations Lifecycle
3. Prompt Management and Versioning
4. Model Selection and Evaluation
5. Observability and Tracing
6. Cost Management
7. Guardrails and Safety
8. Data Pipeline Operations
9. CI/CD for Agentic AI
10. Monitoring in Production
11. Incident Response
12. Tool and Platform Landscape

1. What is LLMOps vs MLOps?

MLOps and LLMOps share the same goal (reliable AI in production) but differ significantly in workflow, tooling, and challenges.

Dimension	Traditional MLOps	LLMOps	Agentic AI Ops
Core artifact	Trained model weights	Prompts + model config	Graphs + prompts + tools
Training	Custom training pipelines	Fine-tuning / RLHF (rare)	Rarely trains; orchestrates
Data dependency	Training data critical	In-context data (RAG)	RAG + tool outputs + state
Evaluation	Metrics (F1, accuracy)	LLM-as-judge, human eval	End-to-end task completion
Latency concern	Batch acceptable	Real-time required	Multi-step chains critical
Cost driver	GPU training compute	API token usage	Multi-call chains compound
Versioning	Model + data versions	Prompt + model versions	Graph + prompt + tool versions
Failure modes	Model drift, data skew	Hallucination, refusal	Tool failures, infinite loops, state corruption

2. The Agentic AI Operations Lifecycle

Agentic AI operations follow a continuous lifecycle that extends traditional MLOps with agent-specific stages.

Phase 1: Design

- Define agent objectives and success criteria
- Design the state graph and tool integrations
- Establish evaluation datasets and benchmarks

Phase 2: Develop

- Build agent graph with LangGraph or CrewAI
- Write and version prompts in a prompt registry
- Implement tools and validate each node independently

Phase 3: Test

- Unit test individual nodes

- Integration test full agent flows
- Run evaluation suites (accuracy, latency, cost)
- Red team for adversarial inputs and edge cases

Phase 4: Deploy

- Containerize and push to registry
- Deploy with health checks and rollback support
- Configure auto-scaling and rate limiting

Phase 5: Monitor

- Track token usage, latency, error rates
- Monitor agent traces end-to-end
- Alert on cost spikes, loop detection, tool failures

Phase 6: Iterate

- Analyze failure traces and update prompts
- A/B test prompt versions
- Update tools and graph topology as requirements evolve

3. Prompt Management and Versioning

Prompts are the primary control interface for LLM behavior. Treat them as code with full version control.

Prompt Registry Architecture

```
# prompt_registry.py
import json, hashlib
from datetime import datetime

class PromptRegistry:
    def __init__(self, storage_path="prompts/"):
        self.storage_path = storage_path

    def register(self, name, template, metadata=None):
        version_hash = hashlib.sha256(
            template.encode()
        ).hexdigest()[:8]
        entry = {
            "name": name,
            "version": version_hash,
            "template": template,
            "metadata": metadata or {},
            "created_at": datetime.utcnow().isoformat(),
        }
        # Save to file or database
        return entry
```

```
def get(self, name, version=None):
    # Retrieve specific version or latest
    pass
```

Best Practices

- **Version every change:** Even a comma change in a prompt can alter behavior.
- **Tag prompts with metadata:** model, temperature, max_tokens, use case.
- **Use parameterized templates:** Avoid hardcoding values inside prompts.
- **Store evaluation results per version:** Link prompt versions to their benchmark scores.
- **Use LangSmith Hub:** Push prompts to LangSmith for team collaboration and A/B testing.

4. Model Selection and Evaluation

Model Selection Framework

Criteria	Weight	Example Metrics
Task accuracy	High	Pass rate on eval suite, F1 on extraction tasks
Latency	High	P50, P95, P99 response times
Cost per call	Medium	Input + output token pricing, calls per task
Context window	Medium	128K vs 200K; matters for RAG-heavy agents
Tool calling quality	High	% correct tool selection, parameter accuracy
Safety and alignment	High	Refusal rate on valid queries, harmful output rate

Evaluation Pipeline

```
# evaluation.py
from langsmith import Client
from langsmith.evaluation import evaluate

client = Client()

def accuracy_evaluator(run, example):
    predicted = run.outputs["response"]
    expected = example.outputs["expected"]
    return {
        "score": 1.0 if predicted.strip() == expected.strip() else 0.0,
        "key": "exact_match"
    }

results = evaluate(
    lambda inputs: agent.invoke(inputs),
    data="agent-eval-dataset",
    evaluators=[accuracy_evaluator],
```

```
)  
    experiment_prefix="gpt-4o-v2"
```

5. Observability and Tracing

For agentic AI, tracing is not optional. Every agent call involves multiple LLM invocations, tool calls, and state transitions. Without tracing, debugging is nearly impossible.

LangSmith Integration

```
# Set environment variables  
export LANGCHAIN_TRACING_V2=true  
export LANGCHAIN_API_KEY=ls-xxx  
export LANGCHAIN_PROJECT=prod-agent-v2  
  
# LangGraph automatically traces all:  
# - LLM calls (input/output/tokens/latency)  
# - Tool invocations (name/input/output/duration)  
# - Graph state transitions  
# - Checkpoints
```

Key Metrics to Track

Metric	Description	Alert Threshold
Token usage per request	Total input + output tokens	> 50K tokens/request
Latency P95	95th percentile response time	> 30 seconds
Error rate	% of requests returning errors	> 2%
Tool failure rate	% of tool calls that fail	> 5%
Loop detection	Agent revisiting same node 3+ times	Any occurrence
Cost per request	Dollar cost including all LLM calls	> \$0.50/request
Hallucination rate	% of responses with factual errors	> 3% (sampled)

6. Cost Management

Agentic AI compounds LLM costs because a single user request can trigger 5-20+ LLM calls. Cost management is critical.

Cost Optimization Strategies

- **Model cascading:** Use cheaper models (GPT-4o-mini, Claude Haiku) for simple tasks, expensive models for complex reasoning.
- **Caching:** Cache identical prompt+input combinations. Use semantic caching for similar queries.
- **Prompt optimization:** Shorter prompts = fewer input tokens. Remove redundant instructions.
- **Recursion limits:** Always set max iterations to prevent runaway loops.
- **Early termination:** Add exit conditions to agent graphs for clear-cut cases.
- **Batching:** Group multiple evaluations into batch API calls (50% cheaper on most providers).
- **Budget alerts:** Set per-project and per-user spending limits with automatic cutoff.

7. Guardrails and Safety

Production agents need multiple layers of safety guardrails to prevent harmful, incorrect, or expensive outputs.

Guardrail Layers

```
# Input guardrails
def input_guard(user_message: str) -> bool:
    # PII detection
    if detect_pii(user_message):
        return False
    # Prompt injection detection
    if detect_injection(user_message):
        return False
    # Topic restriction
    if not is_allowed_topic(user_message):
        return False
    return True

# Output guardrails
def output_guard(agent_response: str) -> str:
    # Redact any leaked PII
    response = redact_pii(agent_response)
    # Fact-check critical claims
    response = verify_facts(response)
    # Toxicity check
    if is_toxic(response):
        return "I cannot provide that response."
    return response
```

NeMo Guardrails Integration

```
# config.yml (Colang)
define user ask about competitors
    "What about [competitor]?"
    "Compare with [competitor]"

define flow
    user ask about competitors
    bot refuse to discuss competitors
    bot suggest focusing on our product
```

8. Data Pipeline Operations

RAG Pipeline Ops

- **Ingestion monitoring:** Track document count, embedding latency, chunking quality.
- **Index freshness:** Automate re-indexing on document updates. Alert on stale indices.
- **Retrieval quality:** Measure recall@k and precision@k continuously.
- **Embedding model versioning:** Track which embedding model generated each vector. Re-embed on model change.

Data Versioning

- **DVC (Data Version Control):** Version large datasets alongside code.
- **Delta Lake / Iceberg:** Version tabular data with time-travel support.
- **Artifact stores:** Use MLflow or Weights & Biases for model and data artifact tracking.

9. CI/CD for Agentic AI

```
# .github/workflows/agent-ci.yml
name: Agent CI/CD
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements.txt

      # Unit tests
      - run: pytest tests/unit/ -v

      # Integration tests (uses test LLM keys)
      - run: pytest tests/integration/ -v
        env:
          OPENAI_API_KEY: ${ secrets.OPENAI_TEST_KEY }

      # Evaluation suite
      - run: python evals/run_evals.py --dataset test-set
        env:
          LANGCHAIN_API_KEY: ${ secrets.LANGSMITH_KEY }

      # Cost check - fail if eval cost exceeds budget
      - run: python evals/check_cost.py --max-cost 5.00

  deploy:
    needs: test
```



```
if: github.ref == 'refs/heads/main'
runs-on: ubuntu-latest
steps:
- run: echo "Deploy to production"
# ... deployment steps
```

10. Monitoring in Production

Dashboard Metrics

- Request volume and throughput (requests/minute)
- Token consumption (input/output/total per hour)
- Cost tracking (hourly, daily, per-user)
- Error rates by type (LLM error, tool error, timeout)
- Agent path distribution (which graph paths are most common)
- User satisfaction signals (thumbs up/down, task completion)

Alerting Rules

Alert	Condition	Severity	Action
Cost spike	> 200% of daily average	High	Investigate and throttle
Error rate surge	> 5% for 5 minutes	Critical	Page on-call engineer
Latency degradation	P95 > 60s for 10 min	Medium	Scale up or check LLM provider
Loop detected	Same node hit 5+ times	High	Kill request, log trace
Token budget exceeded	Project > monthly limit	High	Notify team, throttle non-critical

11. Incident Response

Runbook Template

- **1. Detect:** Automated alert fires (PagerDuty, Opsgenie, Slack alert).
- **2. Assess:** Check LangSmith traces for the failing requests. Identify the failing node/tool.
- **3. Mitigate:** If LLM provider issue, switch to fallback model. If tool failure, disable the tool and use fallback logic.
- **4. Fix:** Deploy prompt or code fix. Run evaluation suite to confirm fix.
- **5. Post-mortem:** Document root cause, add the failing case to the eval dataset, update guardrails.

12. Tool and Platform Landscape

Category	Tools	Notes
Tracing / Observability	LangSmith, Langfuse, Phoenix (Arize), Helicone	LangSmith is tightly integrated with LangGraph
Prompt Management	LangSmith Hub, PromptLayer, Humanloop	Version, test, and deploy prompts
Evaluation	LangSmith Evals, RAGAS, DeepEval, Braintrust	LLM-as-judge + human-in-the-loop
Guardrails	NeMo Guardrails, Guardrails AI, Lakera	Input/output filtering and policy enforcement
Cost Tracking	Helicone, LiteLLM, OpenMeter	Track per-user and per-project spend
Deployment	Railway, Azure Container Apps, AWS Lambda, Modal	Containerized or serverless deployment
Data Versioning	DVC, MLflow, W&B; Artifacts	Track datasets and model artifacts
CI/CD	GitHub Actions, GitLab CI, CircleCI	Automate test + deploy pipelines